Sentence-Level Classification of Cyber Threat Intelligence into MITRE ATT&CK Tactics

AIC211 – AI for Cybersecurity · Project Topic 10: Threat Intelligence / ATT&CK Technique Classification · Team: [add team name and members]

Abstract

Security analysts read threat reports written in free text and must map each described behavior to the MITRE ATT&CK knowledge base. This project builds a multilabel text classifier that reads a single sentence describing attacker behavior and predicts which of the 15 ATT&CK Enterprise tactics (v20.0) it belongs to. Labels are derived from the procedure examples published by MITRE, and the data is split by source group so that near-identical sentences from the same malware family or threat group never appear in both training and test data. A TF-IDF + linear SVM baseline reaches a Macro-F1 of 0.763 on the test set. A fine-tuned DistilBERT model reaches 0.791 Macro-F1, 0.835 Micro-F1 and 0.781 exact match. The improvement is statistically supported by a bootstrap confidence interval. We also analyze the errors, check for leakage through technique names, test how stable the predictions are under mild input noise, and discuss the limits of the data.

1. Problem Statement

Task. Given one sentence from a cyber threat intelligence (CTI) description, predict the set of ATT&CK tactics it expresses. A tactic is the attacker's goal at a stage of an attack (for example Persistence or Credential Access), while a technique is the specific method used to reach that goal.

Formulation. Multilabel text classification at sentence level with 15 labels. The problem is multilabel because one technique can serve several goals: for example, Scheduled Task belongs to Execution, Persistence and Privilege Escalation at the same time. In our data, 13.5% of sentences carry more than one tactic.

Scope. The primary target is tactic classification. We do not classify whole reports, and we do not cover ATT&CK Mobile or ICS. Technique-level classification (top-N techniques) was considered as an extension and is left as future work.

Course requirements met. One baseline and one improved model, a train/validation/test protocol, reproducible code, metrics aligned with the security task (no reliance on accuracy), and a leakage-aware evaluation.

2. Cybersecurity Context

MITRE ATT&CK is the common vocabulary used by defenders to describe attacker behavior. Mapping text to ATT&CK lets a team ask practical questions: which behaviors does this report describe, which of them do our detections already cover, and where are our blind spots. Today this mapping is mostly manual and slow, because CTI reports are long, unstructured and written in many styles.

An automatic classifier does not replace the analyst. It acts as decision support: it proposes tactics for a sentence so that the analyst can confirm or correct them faster. This framing shapes how we judge errors. A missed tactic can leave a behavior unmapped and therefore uncovered by detection logic, while a spurious tactic costs analyst time. We therefore report both precision and recall for every tactic and use Macro-F1 as the main metric.

3. Dataset

3.1 Source

We use **MITRE ATT&CK Enterprise v20.0** in STIX 2.1 format (file enterprise-attack-20.0.json, 26,690 objects), published in the public repository mitre-attack/attack-stix-data. The file contains 706 active techniques and sub-techniques after removing revoked and deprecated entries. Version 20 defines 15 tactics. Compared with older releases, Defense Evasion has been replaced by two tactics, Stealth and Defense Impairment, so our label space differs from many earlier papers.

3.2 Unit of classification and label construction

The unit is a **sentence** (or short passage) taken from the description field of an ATT&CK "uses" relationship, called a procedure example. These sentences have the form "APT29 used PowerShell to ..." and are close to the language of real CTI reports. Each procedure example is linked to one technique. The technique is mapped to its tactics through the kill_chain_phases field of the STIX object, and the sentence inherits all of those tactics as labels. Labels are therefore assigned by MITRE experts rather than by keyword rules.

3.3 Dataset statistics

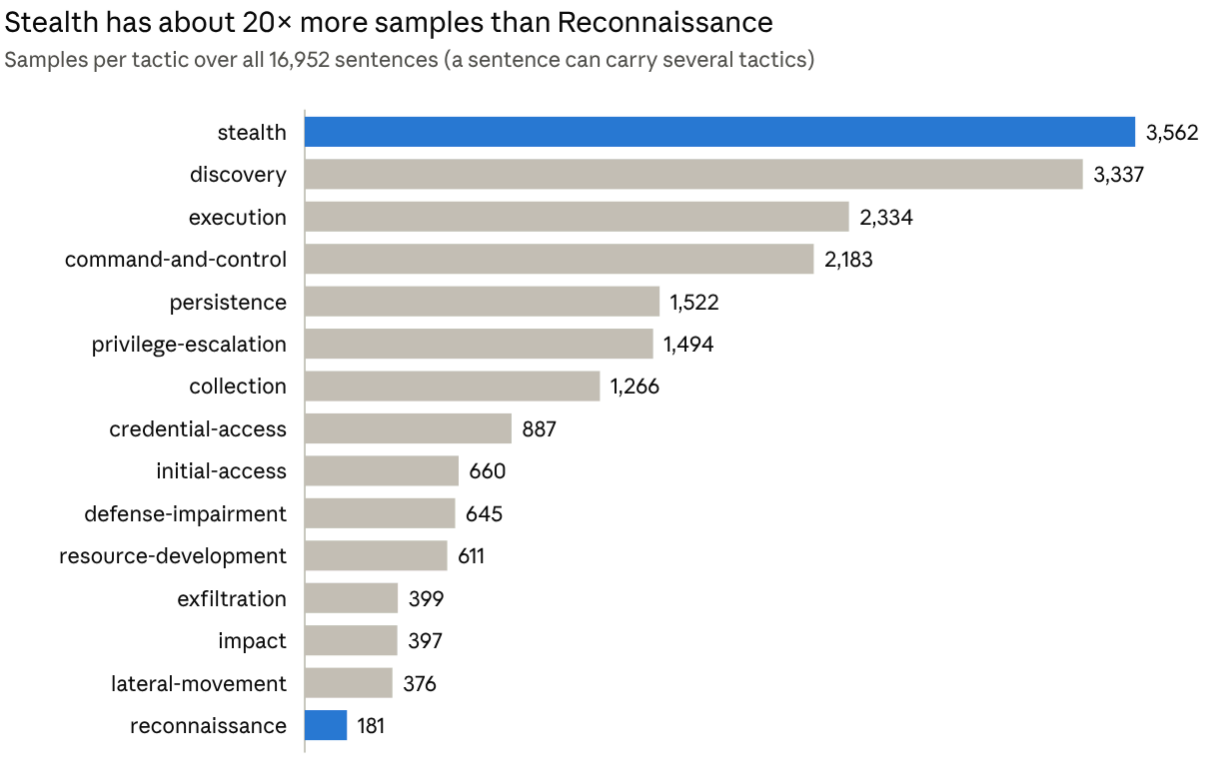
Item	Value
Raw procedure examples	17,636

Item	Value
After removing very short texts (fewer than 5 words) and exact duplicates	17,563
Unique sentences after normalization and merging	16,952
Number of sources (malware, tools, groups, campaigns)	1,071
Average sentence length	about 14.6 words
Sentences with more than one tactic	13.5%
Sentences with more than one technique	0.6%
Sentences that contain the technique name verbatim	13.8%
Sentences that came from more than one source	1.9%

Tactics per sentence: 14,662 sentences have 1 tactic, 1,839 have 2, 293 have 3, 156 have 4, and 2 have 5 or 6.

Number of samples per tactic (all 16,952 sentences, before splitting; a sentence can count for several tactics):

Tactic	Samples	Tactic	Samples
stealth	3,562	initial-access	660
discovery	3,337	defense-impairment	645
execution	2,334	resource-development	611
command-and-control	2,183	exfiltration	399
persistence	1,522	impact	397
privilege-escalation	1,494	lateral-movement	376
collection	1,266	reconnaissance	181
credential-access	887		



Computed from train.csv, val.csv and test.csv · 16,952 sentences, 15 tactics

The distribution is strongly imbalanced: Stealth has about twenty times more examples than Reconnaissance. This is the main reason we use Macro-F1 as the primary metric.

3.4 Dataset justification

The course handout suggests ATT&CK technique descriptions and APTnotes. Raw APTnotes reports have no labels, so they would require manual annotation. ATT&CK procedure examples are already labeled by experts, are numerous, and are written as behavior descriptions, which makes them a practical and reproducible choice. The cost is that these sentences are cleaner and shorter than text in real reports, which we discuss in the limitations.

Since 619 techniques have at least one sample and only 185 of them have 20 or more, technique-level classification would suffer from extremely few examples per class. Tactic-level classification gives every label enough data to learn from.

4. Preprocessing and Feature Engineering

Preprocessing was designed to prevent the model from reading the answer instead of understanding the text.

1. **Remove citations and markup.** Strings such as (Citation: ...) and markdown links are removed.

2. **Remove technique IDs** such as T1059 or T1059.001, which would reveal the label directly.
3. **Mask the actor name.** The name of the group or malware that owns the sentence is replaced with the token <ACTOR>. The model cannot memorize "group X always does technique Y" and the text looks more like a generic report sentence.
4. **Normalize indicators of compromise.** URLs, IP addresses, hashes, registry keys, file paths and e-mail addresses are replaced with tokens such as <URL>, <IP>, <HASH>, <REGKEY>, <PATH> and <EMAIL>. Tool names and commands (powershell, schtasks, ntdsutil) are kept because they are legitimate and strong signals.
5. **Merge duplicates.** Identical sentences from different sources are merged into one row, and their tactic labels are united.
6. **Flag technique-name leakage.** Sentences that contain the technique name verbatim (13.8%) are not deleted, because words like "scheduled task" are natural vocabulary. Instead they are flagged so that results can be reported separately for the harder subset without the name.

Features for the baseline are TF-IDF vectors of word 1–2 grams combined with character 3–5 grams. The transformer uses its own subword tokenizer.

5. Data Splitting Protocol

A random split of sentences would put very similar sentences from the same malware family in both training and test data and would inflate all scores. We therefore split **by group**.

- Every sentence belongs to a group defined by its source. If a sentence has several sources, those sources are merged into a single group using union-find. In total 1,071 sources were merged into 690 groups.
- Groups are assigned to train, validation and test with a greedy algorithm that keeps the tactic distribution close to the target ratio 70/15/15 and gives priority to the rarest tactics.
- Seed 42 is fixed.

Resulting sizes: train 11,752 sentences (69.3%), validation 2,600 (15.3%), test 2,600 (15.3%). For every tactic, the share of its sentences in the test set lies between 14.8% and 17.7%. The rarest tactic, Reconnaissance, has 117, 32 and 32 sentences in train, validation and test.

Checks. No group appears in more than one split. We also measured near-duplicate leakage with TF-IDF cosine similarity: only 0.4% of validation sentences and 0.6% of test sentences have a cosine similarity of 0.9 or higher to some training sentence.

Usage rules. The training set is used for fitting. The validation set is used for choosing models, hyperparameters, thresholds and the best training epoch. The test set was used exactly once, at the end, after all choices had been frozen.

6. Model Design

6.1 Baseline

TF-IDF features (word 1–2 grams, optionally plus character 3–5 grams) with a one-vs-rest linear classifier, so that each tactic has its own binary classifier. We compared Logistic Regression and Linear SVM, both with balanced class weights. If no tactic is predicted for a sentence, the label with the highest score is assigned, because every sentence in the data has at least one tactic. The best baseline on validation data is the **Linear SVM with word + character n-grams**.

6.2 Improved model: fine-tuned DistilBERT

Setting	Value
Pretrained model	distilbert-base-uncased
Output	15 logits with sigmoid (independent probability per tactic)
Loss	Binary cross-entropy with positive weights equal to sqrt(negatives / positives) for each tactic
Optimizer	AdamW, learning rate 5e-5, weight decay 0.01, linear schedule with 10% warmup
Batch size / epochs	32 / 5
Maximum length	128 tokens
Precision	Mixed precision on GPU (Google Colab T4)
Model selection	Epoch with the highest validation Macro-F1

We chose a transformer because the sentences are short and describe behavior in varied wording, a setting where contextual embeddings can generalize beyond keywords. The square-root weighting is a softer form of imbalance handling than fully balanced weights, which in the baseline caused over-prediction for some rare tactics.

6.3 Decision threshold

The default decision threshold is 0.5 for each tactic. We tested per-tactic thresholds tuned on the validation set (see Section 7.3).

7. Evaluation

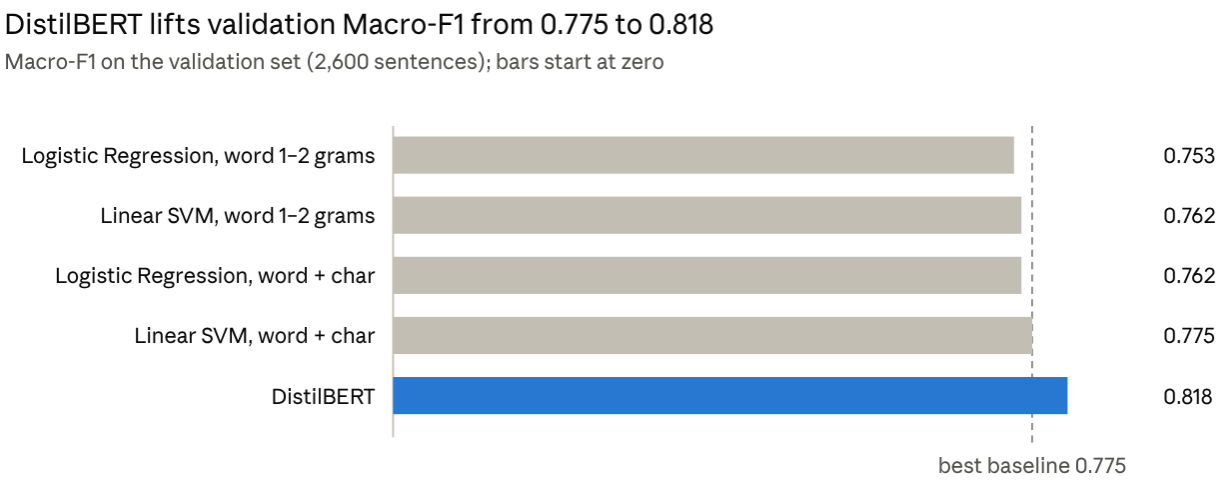
7.1 Metrics and why they fit the security task

- **Macro-F1 (primary).** Averages F1 over the 15 tactics with equal weight. This matters because a rare tactic such as Reconnaissance or Impact is as important to a defender as Discovery. Ignoring it would leave a blind spot.
- **Micro-F1.** Counts every label decision equally and reflects overall performance dominated by large classes.
- **Exact match.** The fraction of sentences whose whole label set is predicted correctly. It is a strict measure for multilabel sentences.
- **Label coverage.** The fraction of tactics predicted correctly at least once. It reached 1.0 for all models, so it does not separate them.
- **Per-tactic precision, recall and F1.** Precision relates to analyst workload from false alarms, recall relates to missed behaviors.

Accuracy is deliberately not used, because with 15 sparse labels a model can obtain a very high accuracy while ignoring rare tactics.

7.2 Validation results

Model	Micro-F1	Macro-F1	Exact match
Logistic Regression, word 1–2 grams	0.797	0.753	0.695
Linear SVM, word 1–2 grams	0.804	0.762	0.706
Logistic Regression, word + char	0.806	0.762	0.712
Linear SVM, word + char (best baseline)	0.811	0.775	0.720
DistilBERT	**0.835**	**0.818**	**0.780**



Validation results from 04_baseline.py and 05_finetune_bert.py · 2,600 sentences

Character n-grams give a small gain of about 0.01 Macro-F1 for the baseline, probably because they capture word variants (exfiltrate, exfiltrated) and tool names. DistilBERT improves Macro-F1 by 0.043 over the best baseline and outperforms it on 13 of the 15 tactics, with the largest gains on rare tactics such as Impact (+0.162), Reconnaissance (+0.103), Resource Development (+0.089) and Exfiltration (+0.087).

7.3 Threshold tuning

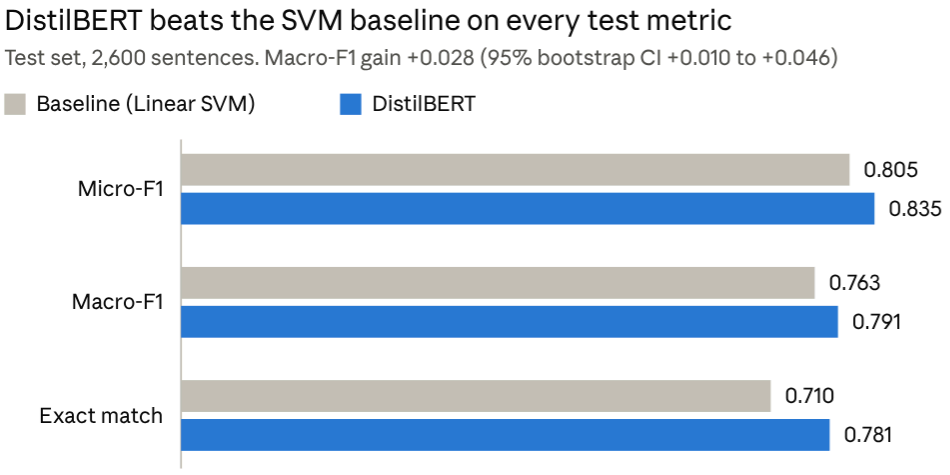
Strategy (DistilBERT, validation)	Micro-F1	Macro-F1	Exact match
Default threshold 0.5	0.835	0.818	0.780
Tuned on all validation data (optimistic)	0.843	0.824	0.807
Tuned with 2-fold grouped cross-validation (honest)	0.834	0.813	0.790

Tuning thresholds and scoring them on the same validation data looks like an improvement, but this estimate is optimistic. When thresholds are tuned on one half of the validation set and evaluated on the other half (folds split by group), Macro-F1 falls below the default (0.813 versus 0.818). The thresholds are therefore not stable, especially for tactics with few examples. **We kept the default threshold 0.5.** Exact match did improve in the honest estimate (0.790), which is a trade-off we note but did not exploit.

7.4 Final test results

The test set was evaluated once with the frozen models.

Model	Micro-F1	Macro-F1	Exact match	Label coverage
Baseline (Linear SVM, word + char)	0.805	0.763	0.710	1.0
DistilBERT	**0.835**	**0.791**	**0.781**	1.0



Test results from 07_final_test.py · 2,600 sentences, evaluated once

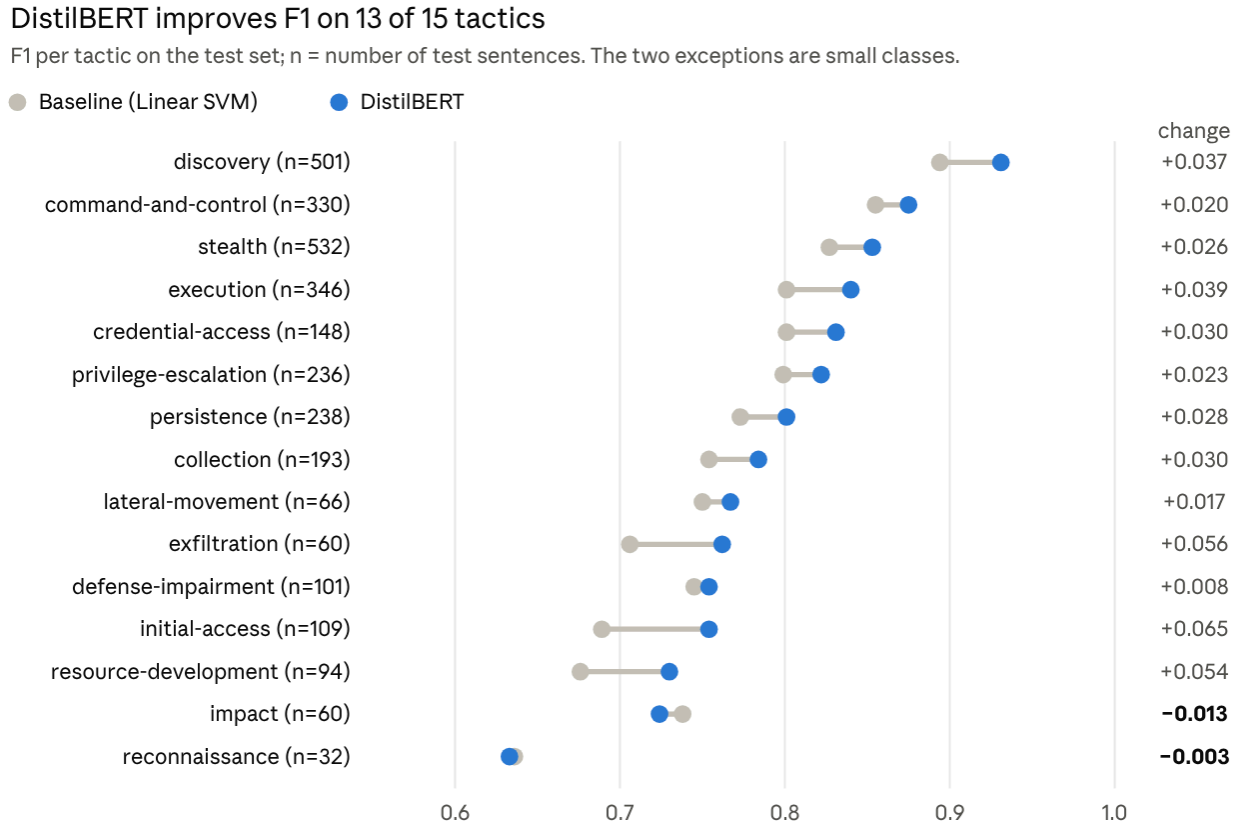
We computed 95% bootstrap confidence intervals (1,000 resamples of test sentences, seed 42). Macro-F1 for DistilBERT is [0.772, 0.807] and for the baseline [0.744, 0.780]. The paired difference is +0.028 with a 95% interval of [+0.010, +0.046], which excludes zero, so the improvement is unlikely to be due to chance. DistilBERT is right and the baseline wrong on 341 sentences, while the reverse occurs on 158 sentences. Counting a sentence as wrong when at least one of its labels is wrong, DistilBERT errs on 570 of 2,600 sentences and the baseline on 753.

Both models score slightly lower on test than on validation (DistilBERT 0.791 versus 0.818 Macro-F1). This is expected, since validation data was used to select the model and epoch, and the test figures are the ones we report.

7.5 Per-tactic results on the test set

Tactic	Support	SVM F1	DistilBERT F1	Difference	DistilBERT precision	DistilBERT recall
collection	193	0.754	0.784	+0.030	0.797	0.772
command-and-control	330	0.855	0.875	+0.020	0.862	0.888

Tactic	Support	SVM F1	DistilBERT F1	Difference	DistilBERT precision	DistilBERT recall
credential-access	148	0.801	0.831	+0.030	0.800	0.865
defense-impairment	101	0.745	0.754	+0.008	0.736	0.772
discovery	501	0.894	0.931	+0.037	0.906	0.958
execution	346	0.801	0.840	+0.039	0.811	0.870
exfiltration	60	0.706	0.762	+0.056	0.727	0.800
impact	60	0.738	0.724	−0.013	0.687	0.767
initial-access	109	0.689	0.754	+0.065	0.723	0.789
lateral-movement	66	0.750	0.767	+0.017	0.761	0.773
persistence	238	0.773	0.801	+0.028	0.799	0.803
privilege-escalation	236	0.799	0.822	+0.023	0.794	0.852
reconnaissance	32	0.636	0.633	−0.003	0.532	0.781
resource-development	94	0.676	0.730	+0.054	0.689	0.777
stealth	532	0.827	0.853	+0.026	0.838	0.868



Test results from 07_final_test.py · per-tactic F1, 2,600 sentences

DistilBERT is better on 13 of 15 tactics. The two exceptions, Impact and Reconnaissance, differ by very little, and both have only 60 and 32 test sentences, so their scores are noisy. Reconnaissance is the weakest tactic: recall is reasonable (0.781) but precision is low (0.532), meaning the model assigns this label too often.

7.6 Check for technique-name leakage

Sentences that contain the technique name verbatim are easier. We evaluated the 2,218 test sentences that do not contain it.

Model	Micro-F1	Macro-F1	Exact match
Baseline, without technique name	0.797	0.743	0.703
DistilBERT, without technique name	0.826	0.770	0.772

Performance on this harder subset is only slightly below the overall performance (DistilBERT Micro-F1 0.826 versus 0.835), and the gap between the two models remains. The models do not rely on reading the technique name.

7.7 Interpretability

The largest positive weights of the word-level Logistic Regression model are meaningful for a security analyst: for example ransomware, wipe and encrypt for Impact; smb, rdp and laterally for Lateral Movement; powershell, wmi and python for Execution. They also reveal a weakness: words such as credentials and phishing appear among the strongest features of both Reconnaissance and Initial Access, which explains part of the confusion between these tactics.

For the fine-tuned model, we built an occlusion-based explanation tool (script 08_explain_demo.py). It removes one word at a time, runs the model again and takes the drop in the predicted probability of the target tactic as that word's importance. Words are shaded in proportion to this drop, and a small web demo lets an analyst paste a paragraph, see the predicted tactics for each sentence and inspect the word importance of any sentence. Occlusion examines one word at a time, so it misses interactions between words, and a high score shows influence on the prediction, not causation.

7.8 Stability under mild input noise

Predicted tactic sets stayed unchanged for 88–93% of sentences under three mild perturbations, and typos were the most harmful. We applied each perturbation to 500 random test sentences (seed 42) and re-ran the frozen DistilBERT model. This is a diagnostic only; nothing was retrained or tuned. The perturbations are: replacing words

with synonyms from a small dictionary (for example “used” by “leveraged”), introducing a typo in about 15% of the words longer than four characters, and appending the benign sentence “The report was published in 2020.”

Input	Label sets unchanged	Micro-F1	Macro-F1
Original	100%	0.875	0.818
Synonym replacement	92.8%	0.855	0.798
Random typos	88.0%	0.831	0.767
Benign sentence appended	91.4%	0.863	0.799

Synonym replacement and the appended sentence lower Macro-F1 by about 0.02, typos by 0.05. Typos probably hurt most because subword tokenization splits misspelled technical terms, such as tool names and commands, into meaningless pieces.

Three cautions apply. The scores are not comparable with the main test result (Macro-F1 0.791 on 2,600 sentences), because this is a random subset of 500 sentences; compare the rows within the table only. The synonym dictionary is small, so many sentences contain no replaceable word and that row is optimistic. Finally, these are mild random changes, not a targeted attack: an adversary who knows the model could choose words far more effectively, and we did not test that.

8. Error Analysis

The analysis below uses validation and test predictions of DistilBERT. Both give the same picture.

Sentences with several tactics are harder. On the test set, exact match is 0.798 for sentences with one tactic, 0.655 for two and 0.628 for three. The baseline is much weaker for the 4-tactic group (0.424 versus 0.818 for DistilBERT), although this group has only 33 sentences.

Longer sentences are harder. Exact match on the test set is 0.861 for sentences of up to 10 words, 0.775 for 11–20 words and 0.641 for 21–40 words. Longer passages often combine several ideas but carry a label for only one technique. Only 40 test sentences have more than 40 words, so that group is not reliable.

Most confusions occur inside one family of tactics. The most frequent test confusions are Execution predicted as Stealth (16 cases), Collection predicted as Discovery (12), Execution predicted as Privilege Escalation (9), Stealth predicted as Execution (9) and Privilege Escalation predicted as Execution (8). Many techniques legitimately belong to

several of these tactics, and the boundary between hiding from detection (Stealth) and disabling protection (Defense Impairment) is naturally blurry. On the validation set, Lateral Movement had the lowest recall (0.61).

Many confident errors reflect label noise, not model failure. Labels belong to the technique, not to the individual sentence. Examples from the validation set:

- "<ACTOR> has abused peer connections to exfiltrate data." has the label Defense Impairment, while the text describes exfiltration. The model predicted Command and Control and Exfiltration, which reads as more reasonable than the official label.
- "<ACTOR> has executed the Windows-native vssadmin command to create volume shadow copies." has the label Stealth, and the model predicted Execution.
- "<ACTOR> has attempted to collect browser information." has the label Collection, and the model predicted Discovery.

As a result, the maximum achievable score on this data is below 100%, and part of the remaining error is inherent to how the labels are built.

9. Limitations

1. **Labels are technique-level, not sentence-level.** Each sentence inherits all tactics of its technique even if it describes only one aspect, which creates label noise (Section 8).
2. **Domain gap.** All sentences come from ATT&CK itself, which is concise and edited. Real CTI reports contain longer passages, narrative text, tables and ambiguous statements. We did not evaluate on raw reports such as APTnotes with manually labeled sentences, so generalization to real reports is untested.
3. **Small samples for rare tactics.** Reconnaissance has 32 test sentences and Impact and Exfiltration 60, so per-tactic F1 values are uncertain, and our conclusions about those tactics are weak.
4. **Single split and single run.** Results come from one group-based split and one training run with a fixed seed. Other seeds may change the scores by a small amount, and we did not average over several runs.
5. **Coverage of MITRE examples.** The data covers behaviors that MITRE has documented, which favors well-known groups and malware families.
6. **Tactic level only.** We did not classify the 600+ techniques and sub-techniques.
7. **Limited model comparison.** We tested one transformer (DistilBERT) and did not test security-specific models.
8. **Limited robustness testing and no rejection option.** We tested only mild random noise (Section 7.8) and did not test targeted adversarial rewording or deliberately obfuscated text. The classifier also always returns at least one tactic, because every

training sentence has one, so text unrelated to attacker behavior still receives a tactic; a “no tactic” class or a minimum confidence would be needed in practice.

10. Ethical Considerations

- **Dual use.** The model classifies descriptions of attacker behavior. It does not generate attacks, and no exploit code or operational attack instructions are produced. Still, a tool that understands attacker techniques could be used by an adversary to study which behaviors defenders commonly detect. We use only public MITRE data and recommend using the model for defense.
- **Public data.** ATT&CK is published under MITRE’s terms of use (attribution is required), and we cite it. We did not collect any private or sensitive reports. If the model is later applied to internal incident reports, those documents may contain confidential data and must be handled according to organizational policy.
- **Human in the loop.** Predictions can be wrong, as the error analysis shows. The system should support analysts rather than make autonomous security decisions, and low-confidence predictions should be reviewed by a person.
- **Bias in coverage.** Because the labels come from documented behaviors of known actors, the model may under-recognize behaviors of lesser-known actors or novel techniques, which could give a false sense of coverage.
- **Adversarial evasion.** An attacker who controls the wording of a report or log message could try to mislead a text classifier. The model should not be the only control in a security process.

11. Conclusion and Future Work

We built a reproducible pipeline that maps single CTI sentences to 15 ATT&CK Enterprise tactics. With group-aware splitting and leakage controls, a TF-IDF + Linear SVM baseline reaches 0.763 Macro-F1 on test data, and a fine-tuned DistilBERT reaches 0.791 Macro-F1, 0.835 Micro-F1 and 0.781 exact match. The improvement is supported by a bootstrap interval, it appears on 13 of 15 tactics, and it holds on sentences that do not contain the technique name. Per-tactic threshold tuning did not help when estimated honestly, so we kept the default threshold. Under mild random noise the predictions stayed stable for 88–93% of sentences, and typos were the most harmful change. Remaining errors are concentrated among overlapping tactics, on long multi-idea sentences, and in sentences whose official labels are noisy.

Future work.

- Evaluate on real CTI reports (for example APTnotes) with a small manually labeled test set.

- Extend to technique-level classification for the most frequent techniques, and use hierarchical prediction (tactic first, then technique).
- Compare with security-specific language models and with larger transformers, and average over several seeds.
- Compare the occlusion explanations with LIME or SHAP and evaluate them with analysts.
- Test robustness against targeted adversarial rewording, and add a “no tactic” option or a minimum confidence.

Appendix A: Reproducibility

All code is organized as numbered scripts. Running them in order reproduces the results (seed 42).

Script	Purpose
01_build_dataset.py	Extract sentences, techniques and tactics from ATT&CK v20.0 STIX
02_preprocess.py	Clean text, mask actors and indicators, merge duplicates
03_split_data.py	Group-aware, multilabel-stratified 70/15/15 split with leakage checks
04_baseline.py	TF-IDF + Logistic Regression / Linear SVM baselines (validation only)
05_finetune_bert.py	Fine-tune DistilBERT (GPU recommended, e.g. Google Colab)
06_threshold_and_errors.py	Threshold tuning with honest cross-validation, validation error analysis
07_final_test.py	Single final evaluation on the test set with bootstrap confidence intervals
08_explain_demo.py	Occlusion-based word importance, stability test under input noise and a small web demo

Environment: Python 3, pandas, scikit-learn, scipy, PyTorch and Hugging Face Transformers.

References

- MITRE ATT&CK Enterprise, version 20.0, STIX 2.1 data from the repository mitre-attack/attack-stix-data.
- Sanh, V., Debut, L., Chaumond, J., Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.
- Devlin, J., Chang, M., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research.